A Atlas One

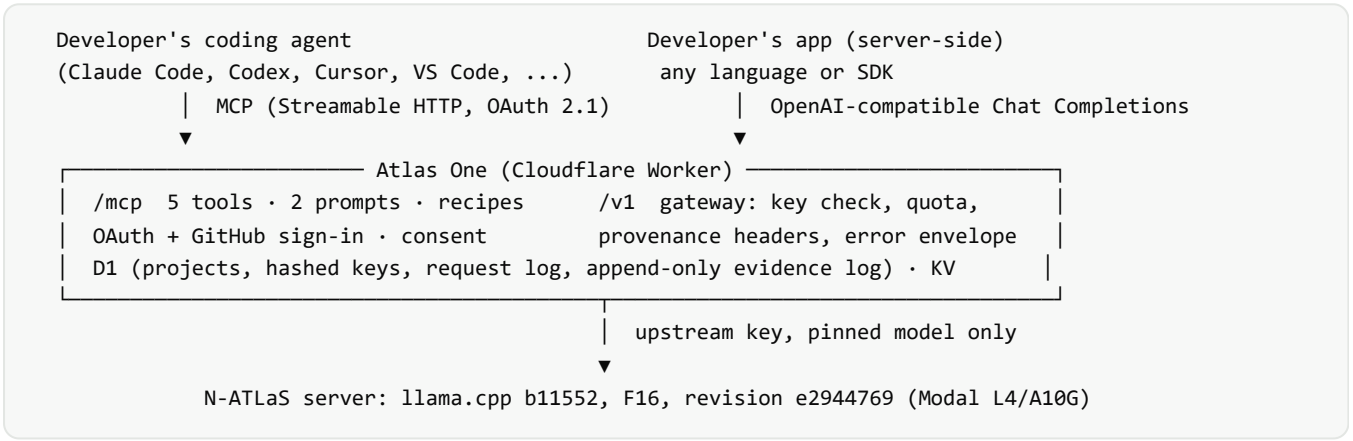
NAIC 2026 · ITEM 4

Technical Documentation

Architecture, setup and usage of Atlas One, sufficient for an independent party to integrate with it or run their own.

Product	Date	Live	Source
Atlas One v1.0	12 October 2026	atlas-one...workers.dev	github.com/agapemiteu/atlasone

1. Architecture



COMPONENT	TECHNOLOGY	ROLE
Worker	Cloudflare Workers, TypeScript	MCP server, OAuth, gateway, website and evidence pages, one deployment
MCP	Model Context Protocol, Streamable HTTP (stateless), protocol 2025-03-26 to 2025-11-25	5 tools, 2 prompts, recipe resources, output schemas, annotations
Auth	OAuth 2.1 + PKCE, dynamic client registration, client ID metadata documents; GitHub sign-in	Agents sign in once; consent page with scopes; personal tokens for CI
Data	D1 (SQLite), KV	Projects, hashed keys, claims, quotas, request log, append-only evidence log; OAuth state
Model server	llama.cpp b11552, F16 N-ATLaS, Modal (L4/A10G) or Kaggle (2× T4)	OpenAI-compatible inference, woken on demand, sleeps when idle
Website	React 19, Vite, Tailwind, shadcn/ui	Home, guide, docs; served as static assets by the same Worker

2. Using Atlas One

With a coding agent

AGENT	CONNECT
Codex	`codex mcp add atlas-one --url https://atlas-one.collins-coordinator-worker.workers.dev/mcp` (signs in automatically)
Claude Code	`claude mcp add --transport http atlas-one https://atlas-one.collins-coordinator-worker.workers.dev/mcp`, then /mcp → Authenticate
Cursor, VS Code, Windsurf, Gemini CLI, others	Add the same URL as a remote MCP server (configs in the docs)

Then ask in plain words, for example: "*Build a one-page app that translates English into Hausa, Igbo or Yoruba using N-ATLaS through Atlas One. Keep the key server-side, then test and verify it.*" The agent follows a four-call path:

CALL	DOES
`atlas_discover`	Live model state, model card, limits, licence, projects; wakes a sleeping model
`atlas_bind_project`	Creates the project; returns a one-time key link (the developer confirms in the browser) and the integration recipe
`atlas_test` inference	Genuine inference with provenance; returns the app-check nonce
`atlas_test` verify_app	Proves the app's own route reached N-ATLaS; issues a signed receipt
`atlas_inspect`	Requests, latency p50/p95, recent errors with request ids, quota (diagnosis)
`atlas_feedback`	Records the developer's own rating when the agent asks at a natural pause

Prompts `build_with_n_atlas` and `diagnose_n_atlas` appear as commands in clients that support them. An agent skill (standard SKILL.md) is published at `https://atlas-one.collins-coordinator-worker.workers.dev/skill.md`.

Directly with the API

Sign in at `https://atlas-one.collins-coordinator-worker.workers.dev/access`, Create API key, then call the OpenAI-compatible endpoint from your server:

```
curl https://atlas-one.collins-coordinator-worker.workers.dev/v1/chat/completions \
  -H "authorization: Bearer $ATLAS_ONE_API_KEY" -H "content-type: application/json" \
  -d '{"model":"n-atlas-8b","messages":[
        {"role":"system","content":"Reply in Hausa."},
        {"role":"user","content":"What is the capital of Nigeria?"}]}'
```

Any OpenAI SDK works by changing the base URL and key. Deploy by setting `ATLAS_ONE_BASE_URL` and `ATLAS_ONE_API_KEY` as server-side variables on any host, then verify the live URL the same way.

3. Errors and limits

CODE	HTTP	RETRY	MEANING
AUTH_REQUIRED / INVALID_KEY / KEY_REVOKED	401	No	Missing, wrong or revoked key
VALIDATION_FAILED / REQUEST_TOO_LARGE	400 / 413	No	Bad request or body over 64 KB

CODE	HTTP	RETRY	MEANING
RATE_LIMITED	429	Yes	Project quota: 30 per minute, 500 per day
BACKEND_WARMING / BACKEND_BUSY	503 / 429	Yes	Model starting (about 30 s) or all slots busy; retry after retry_after_s
MODEL_OFFLINE / INFERENCE_TIMEOUT / UPSTREAM_ERROR	503 / 504 / 502	Yes	Model unreachable, slow (120 s) or failed; never replaced by another model
INVALID_OUTPUT	502	No	The model returned an empty or unusable completion

Errors share one envelope: `{ "error": { "code", "message", "retryable", "retry_after_s", "request_id" } }`. Limits: max_tokens 1,024; 4,096 tokens context per request.

4. Security

- Keys are created only after the owner confirms in the browser, shown once, stored as hashes, never returned to an agent or written to logs.
- Agents receive only the scopes approved on the consent page; any agent can be signed out from `/access`.
- Every browser form is CSRF-protected and bound to the signed-in session; pages send strict security headers (CSP, frame denial).
- The model server accepts only the gateway's private upstream key; only the active provider's hosts may register.
- The evidence log stores metadata only; a test enforces that prompts, outputs and keys never reach it.

5. Run your own

```
git clone https://github.com/agapemiteu/atlasone && cd atlasone
npm install && npm install --prefix web
npx wrangler d1 create atlas-one && npx wrangler kv namespace create OAUTH_KV   # ids into wrangler.jsonc
npx wrangler d1 migrations apply atlas-one --remote
npx wrangler secret put <NAME>   # ATLAS_UPSTREAM_KEY, ATLAS_ADMIN_KEY, SESSION_SECRET,
                                 # GITHUB_CLIENT_SECRET, HF_TOKEN, RECEIPT_SIGNING_KEY (Ed25519 JWK)
modal deploy inference/modal_serve.py   # Modal secret "atlas-one": HF_TOKEN, ATLAS_UPSTREAM_KEY
npm run deploy                          # builds the website, deploys the Worker
```

Set `COMPUTE` to `modal` (stable URL, about 30 s cold start) or `kaggle` (free, can queue), register the model server once, and create a GitHub OAuth App with callbacks `/auth/github/mcp` and `/auth/github/web`. Full steps: `docs/ARCHITECTURE.md`.

6. Quality

CHECK	RESULT
Unit and integration tests	136 Worker tests run inside workerd with real local D1 and KV (MCP conformance in both protocol eras, OAuth through the real entry point, every failure mode, security, evidence signing); 6 web tests
Live end-to-end	Gateway 5/5 (official OpenAI SDK), MCP flow in both protocol eras including verify_app
Evidence	Signed snapshot verified 6/6

Repository map: `src/` Worker (index, mcp, tools, lib, routes, web) · `web/` website · `inference/` model servers · `migrations/` schema · `test/` tests · `scripts/` end-to-end checks and evidence tools · `docs/` architecture, evidence, PRD.